SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: CSA1404 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. *(BL3)*

Assessment Tool Weightage: 10%

QUESTIONS: **Total Marks:50**

Annexure A

TECHNICAL PRESENTATION

Code of Conduct:

*I **CHINTHAPARTHI NAVEEN REDDY / 192525206**_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

C. Naveen Reddy
Signature of the student:

Annexure A

TECHNICAL PRESENTATION

Q1. Compiler Phases and Their Functions

Understanding of Case

A. Explain the different phases of a compiler, including lexical analysis, syntax analysis, semantic analysis, intermediate code generation, optimization, and code generation.

B. Describe how each phase contributes to converting source code into machine-level instructions.

Application of Concepts

C. Apply compiler phases to identify errors such as invalid tokens, syntactical mistakes, and semantic inconsistencies.

D. Illustrate phase-wise error detection using appropriate C program examples.

Analysis

E. Examine how each phase relies on the output of the previous phase for correct processing.

F. Analyze how errors in early stages (lexical and syntax) can halt further compilation.

Solution Design

G. Propose improved error recovery strategies across different compiler phases.

H. Suggest a modular compiler architecture to enhance flexibility and debugging.

Clarity

I. Represent the sequence of compiler phases using flowcharts.

J. Provide step-by-step illustrations showing how source code is processed.

Q2. Lexical Analysis and Token Specification using LEX

Understanding of Concept

A. Define tokens, lexemes, and patterns in the context of lexical analysis.
B. Explain the function of a lexical analyzer in scanning and processing source code**.**

Application of Concepts

C. Demonstrate token generation using the LEX tool with suitable input examples.
D. Apply regular expressions to recognize identifiers, constants, and operators.

Analysis

E. Examine common lexical errors such as invalid identifiers and illegal symbols.
F. Analyze how proper tokenization enhances the efficiency of parsing.

Solution Design

G. Develop token recognition rules using regular expressions in LEX.
H. Incorporate mechanisms for handling lexical errors effectively.

Clarity

I. Provide clear LEX syntax examples for better understanding.
J. Illustrate token generation using input-output examples.

Q3. Language Translators: Compiler vs Interpreter

Understanding of Case

A. Differentiate between a compiler and an interpreter based on their method of program execution.
B. Explain how each type of language processor converts high-level code into executable form.

B. **Application of Concepts**

C.Applies concepts to languages like C (compiler) and Python (interpreter).

D.Demonstrates how errors are handled differently in both.

Analysis

E.Compares performance, speed, and memory usage of both processors.

F.Evaluates advantages and disadvantages in real-world scenarios.

Solution Design

G.Suggests using compilers for performance-critical applications.

H.Recommends interpreters for rapid development and debugging.

Clarity

I.Uses comparison tables for easy understanding.

J.Includes real-life examples and execution flow diagrams.

Q4.Organization of Compiler Phases and Code Optimization

Understanding of Case

A.Explains grouping into front-end (analysis) and back-end (synthesis).

B. Describes the role of each group in compilation.

Application of Concepts

C.Applies grouping to simplify compiler design and debugging.

D.Demonstrates how front-end handles errors and back-end improves performance.

Analysis

E.Analyzes how grouping improves modularity and maintainability.

F.Evaluates optimization techniques like constant folding and dead code removal.

Solution Design

G.Designs efficient compiler structure using grouped phases.

H.Suggests advanced optimization techniques for better performance.

Clarity

I.Uses diagrams to represent front-end and back-end clearly.

J.Provides before-and-after code examples for optimization.

Q5. Compiler Design Tools: LEX and YACC

Understanding of Case

A.Explains the purpose of LEX (scanner) and YACC (parser).

B. Describes how these tools automate compiler construction.

Application of Concepts

C. Demonstrates building a simple compiler using LEX and YACC.

D.Applies tokenization and parsing using real examples.

Analysis

E.Analyzes benefits like reduced development time and improved accuracy.

F.Evaluates limitations such as complexity in debugging.

Solution Design

G.Designs a step-by-step approach to build a compiler using tools.

H.Suggests improvements like better error handling and integration.

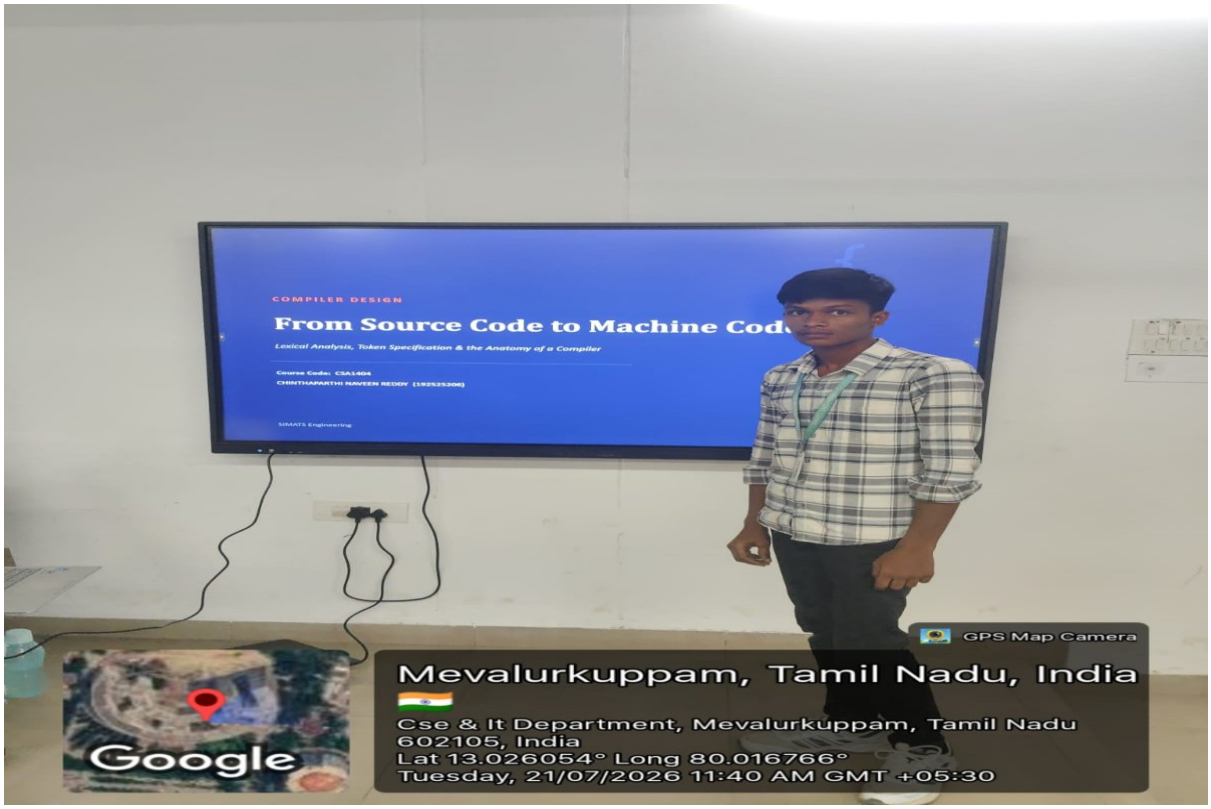
Clarity

I.Presents workflow diagrams (LEX → YACC → Output).

J.Provides clear step-by-step execution examples.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand



COMPILER DESIGN

From Source Code to Machine Code

Lexical Analysis, Token Specification & the Anatomy of a Compiler

Course Code: CSA1404
CHINTHAPARTHI NAVEEN REDDY (192525206)

SIMATS Engineering

TOPIC 1 · COMPILER PHASES

Six Compiler Phases

1. Lexical Analysis
Source → token stream

2. Syntax Analysis
Tokens → parse tree

3. Semantic Analysis
Type & scope checking

4. Intermediate Code
Generates IR (e.g. 3-address code)

5. Code Optimization
Improves IR efficiency

6. Code Generation
IR → target machine code

TOPIC 1 · COMPILER PHASES

Introduction to Compiler

A compiler is a program that translates source code written in a high-level language into equivalent target code (machine or assembly language), while preserving the meaning of the program.

Input
High-level source program (C, Java, Python, etc.)

Output
Semantically equivalent target / machine code

Role
Bridges human-readable code and machine-executable instructions

Why We Need Compilers

- Humans write in high-level, readable syntax
- Machines execute only binary instructions
- Compilers ensure correctness, speed, and portability
- Enable reusable, optimized, error-checked programs

TOPIC 1 · COMPILER PHASES

Error Detection in Each Phase

Compiler Phase	Typical Errors Detected
Lexical Analysis	Illegal characters, malformed tokens
Syntax Analysis	Grammar violations, missing symbols
Semantic Analysis	Type mismatches, undeclared variables
Intermediate Code	Inconsistent operand / operation errors
Optimization	Unsafe transformations changing meaning
Code Generation	Register/memory allocation failures

TOPIC 1 · COMPILER PHASES

Advantages of Modular Compiler Design

Separation of Concerns
Each phase handles one well-defined task, simplifying design

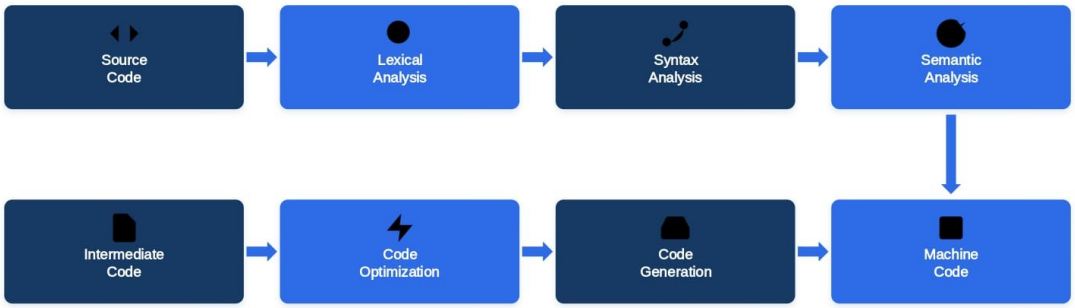
Easier Maintenance
Phases can be debugged or upgraded independently

Reusability
Front-end / back-end modules can be reused across compilers

Reliable Error Isolation
Errors are localized to the phase that produced them

TOPIC 1 · COMPILER PHASES

Compiler Phase Flowchart



Each stage passes its output as input to the next, transforming source text into executable machine instructions.

TOPIC 1 · CONCLUSION

Compiler Phases and Their Functions

- A compiler transforms high-level source code into machine code through six well-defined phases.
- Each phase performs a specific role — from lexical scanning to final code generation.
- Errors are detected locally at each phase, simplifying debugging.
- Modular design improves maintainability, reusability, and reliability of the compiler.

TOPIC 2 · LEXICAL ANALYSIS & LEX

Introduction to Lexical Analysis

Lexical analysis is the first phase of a compiler. It reads the source program character-by-character and groups characters into meaningful sequences called tokens.

Scans
Reads raw source text stream

Groups
Combines characters into lexemes

Classifies
Assigns a token type to each lexeme

Forwards
Sends token stream to the parser

TOPIC 2 · LEXICAL ANALYSIS & LEX

Tokens, Lexemes, Patterns

Token
A category / class representing a set of strings (e.g. identifier, keyword, operator)

Lexeme
The actual character sequence in source code matching a token (e.g. "count", "123")

Pattern
The rule (often a regular expression) describing the form of lexemes for a token

Example

```
int  count  =  10;
```

- Token: KEYWORD → Lexeme: "int"
- Token: IDENTIFIER → Lexeme: "count"
- Token: ASSIGN_OP → Lexeme: "="
- Token: NUMBER → Lexeme: "10"
- Pattern for IDENTIFIER: letter (letter|digit)*

TOPIC 2 · LEXICAL ANALYSIS & LEX

Regular Expressions in LEX

LEX uses regular expressions to specify token patterns declaratively — the tool then generates a scanner (lexical analyzer) automatically.

Regular Expression	Meaning
`[a-zA-Z_][a-zA-Z0-9_]*`	Identifier: letter/underscore followed by letters, digits, underscores
`[0-9]+`	Integer constant — one or more digits
`[0-9]+\.[0-9]+`	Floating-point constant
`"([^"\\n])*"`	String literal enclosed in double quotes
`\+|\-|\*|\/`	Arithmetic operators
`\/\/.*`	Single-line comment

TOPIC 2 · LEXICAL ANALYSIS & LEX

Lexical Error Handling

Illegal Character
A symbol not defined in the language's alphabet (e.g. "@" in an identifier)

Malformed Token
Input that does not match any defined pattern (e.g. "12.3.4")

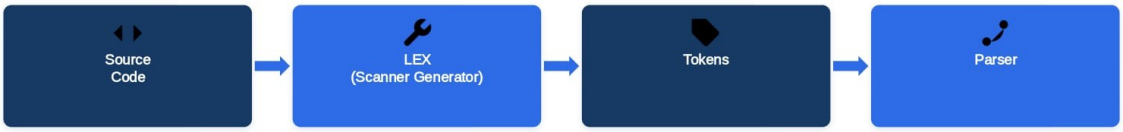
Unmatched Input
Characters left over that no rule in the LEX specification recognizes

Recovery Strategies

- Panic mode: delete characters until a valid token starts
- Insert a missing character to complete a token
- Replace an invalid character with a valid one
- Report line number and nature of the error to the user
- Continue scanning to detect further errors in one pass

TOPIC 2 · LEXICAL ANALYSIS & LEX

LEX Workflow Diagram



LEX reads a .l specification file containing regular-expression rules, then generates a C program (lex.yy.c) that performs scanning.

TOPIC 2 · CONCLUSION

Lexical Analysis and Token Specification using LEX

- Lexical analysis converts source text into a stream of classified tokens.
- Tokens, lexemes, and patterns together define the vocabulary of a language.
- LEX automates scanner construction using regular expressions.
- Robust error handling lets the scanner recover and continue after invalid input.

TOPIC 3 · COMPILER VS INTERPRETER

Introduction to Language Translators

A language translator converts a program written in a source language into another language. The two most common types are compilers and interpreters, each translating and executing code differently.

Compiler
Translates the entire program before execution

Interpreter
Translates and executes line-by-line, on the fly

Assembler
Converts assembly language to machine code

TOPIC 3 · COMPILER VS INTERPRETER

Compiler

Translates the entire source program into machine code in one pass, producing a standalone executable file before any execution begins.

Fast Execution

Runs directly as native machine code once compiled

Separate Stages

Compilation and execution occur as distinct steps

Batch Errors

All errors are reported together after full compilation

Examples

C, C++, Rust, Go

TOPIC 3 · COMPILER VS INTERPRETER

Interpreter

Translates and executes source statements one at a time, without producing a separate standalone executable file.

Line-by-Line

Reads, translates and runs one statement at a time

Immediate Errors

Stops and reports an error as soon as it is found

No Executable

Requires the interpreter present at every run

Examples

Python, JavaScript, Ruby

TOPIC 3 · COMPILER VS INTERPRETER

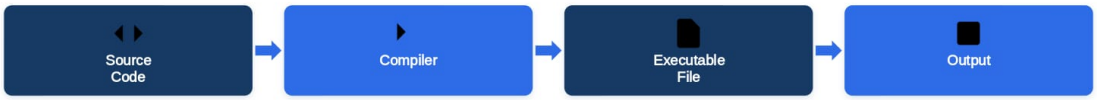
Comparison & Real-Life Examples

Aspect	Compiler (C)	Interpreter (Python)
Translation	Whole program at once	Statement by statement
Speed	Faster execution (pre-compiled)	Slower (translated at runtime)
Output	Standalone executable (.exe)	No separate executable file
Error Reporting	After entire compilation	Immediately, line by line
Typical Use	System / performance-critical software	Scripting, rapid prototyping

TOPIC 3 · COMPILER VS INTERPRETER

Execution Flow Diagram

Compiler Execution



Interpreter Execution



TOPIC 3 · CONCLUSION

Language Translators: Compiler vs Interpreter

- Compilers translate the whole program before execution; interpreters translate and run line-by-line.
- Compiled programs execute faster; interpreted programs are easier to debug interactively.
- C is a classic compiled language; Python is a classic interpreted language.
- The choice between them depends on performance needs versus development flexibility.

TOPIC 4 · ORGANIZATION & OPTIMIZATION

Front-End Compiler

The front-end is source-language dependent. It analyzes the source program, checks correctness, and produces an intermediate representation independent of the target machine.

Lexical Analysis

Tokenizes source input

Syntax Analysis

Builds the parse tree

Semantic Analysis

Type & scope checking

IR Generation

Produces intermediate code

TOPIC 4 · ORGANIZATION & OPTIMIZATION

Back-End Compiler

The back-end is target-machine dependent. It takes the intermediate representation, optimizes it, and generates final code for the target architecture.

Code Optimization

Improves IR efficiency

Register Allocation

Assigns variables to registers

Code Generation

Emits target machine code

Machine Code

Final executable output

TOPIC 4 · ORGANIZATION & OPTIMIZATION

Code Optimization Techniques

Code optimization improves intermediate or target code to use less time, memory, or power — without changing the program's output.

Constant Folding

Evaluate constant expressions at compile time

Dead Code Elimination

Remove code that never affects output

Loop Optimization

Reduce redundant work inside loops

Common Subexpression Elimination

Reuse previously computed values

TOPIC 4 · ORGANIZATION & OPTIMIZATION

Constant Folding & Dead Code Elimination

Constant Folding

Evaluates constant expressions at compile time instead of at runtime.

```
Before:  x = 4 * 5;
After:    x = 20;
```

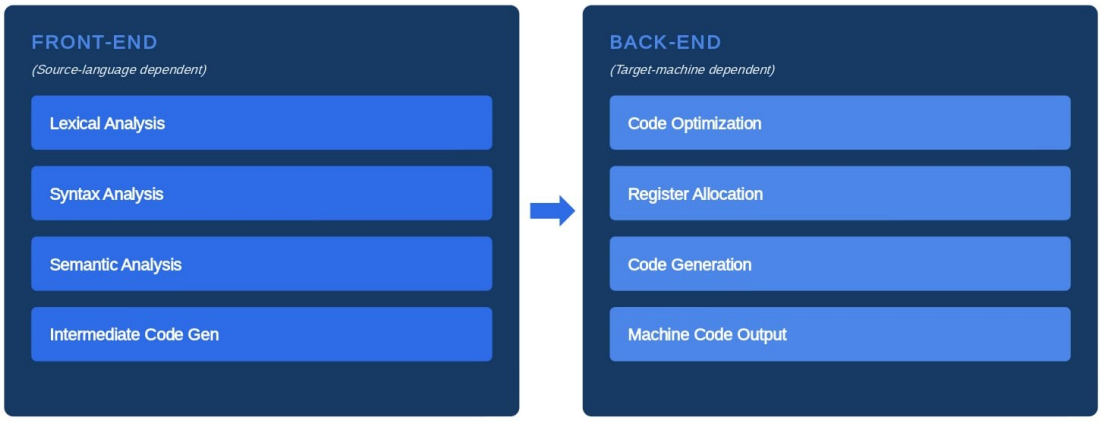
Dead Code Elimination

Removes statements whose results are never used, reducing code size.

```
int x = 5;
int y = 10;   // y unused
return x;
```

TOPIC 4 · ORGANIZATION & OPTIMIZATION

Front-End / Back-End Architecture



TOPIC 4 · CONCLUSION

Organization of Compiler Phases and Code Optimization

- The front-end handles source-language analysis; the back-end handles target-code generation.
- This split lets one front-end pair with multiple back-ends for different machines.
- Optimization techniques like constant folding and dead code elimination improve performance.
- Together, front-end and back-end organization makes compilers portable and efficient.

TOPIC 5 · LEX AND YACC

Introduction to LEX

LEX (Lexical Analyzer Generator) automatically creates a lexical analyzer from a set of regular-expression rules written in a .l specification file.

Rule-Based

Patterns defined using regular expressions

Generates C Code

Outputs lex.yy.c, a ready scanner program

Produces Tokens

Emits classified tokens for the parser

Fast Scanning

Uses efficient finite-automaton matching

TOPIC 5 · LEX AND YACC

Introduction to YACC

YACC (Yet Another Compiler Compiler) generates a parser from a grammar specification, building a parse tree and checking syntax against grammar rules.

Grammar-Based

Rules defined in Backus-Naur-like syntax

Generates C Code

Outputs y.tab.c, a ready parser program

Builds Parse Tree

Validates structure of the token stream

Detects Syntax Errors

Reports grammar violations precisely

TOPIC 5 · LEX AND YACC

LEX and YACC Working Together

LEX and YACC integrate tightly: LEX supplies tokens on demand, and YACC drives the process, calling LEX's yylex() function each time it needs the next token.

1. YACC parser requests the next token by calling yylex()
2. LEX scanner matches the input and returns a token
3. YACC matches tokens against its grammar rules
4. On a full match, YACC executes the associated action / builds the parse tree

TOPIC 5 · LEX AND YACC

Advantages and Limitations

Advantages

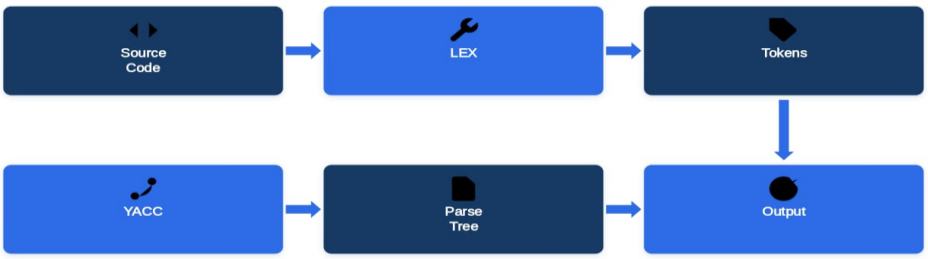
- Automates tedious, error-prone scanner/parser coding
- Produces efficient, well-tested C code
- Grammar and rules are easy to modify and extend
- Widely used, well-documented, portable tools

Limitations

- Generated code can be harder to debug than hand-written code
- Limited flexibility for highly irregular grammars
- Error messages from generated parsers can be cryptic
- Learning curve for grammar/specification syntax

TOPIC 5 · LEX AND YACC

LEX + YACC Workflow



TOPIC 5 · CONCLUSION

Compiler Design Tools: LEX and YACC

- LEX automates lexical analyzer generation from regular-expression rules.
- YACC automates parser generation from grammar rules, building the parse tree.
- Together they form a complete, efficient front-end pipeline for a compiler.
- Despite some limitations, they remain foundational tools in compiler construction.